CCS231 - Parallel and Distributed Computing

System Design Document

Distributed Publish-Subscribe Messaging Middleware

Abstract

This document outlines the architecture and design decisions of a centralized Publish-Subscribe (Pub/Sub) distributed messaging system. Implemented in Python, the middleware features persistent message queues backed by an optimized SQLite database, guaranteeing at-least-once delivery for disconnected clients. It utilizes raw TCP sockets with custom message framing for high-performance Inter-Process Communication (IPC). Furthermore, the architecture seamlessly integrates a decoupled web dashboard that bridges synchronous TCP feeds with asynchronous WebSockets to visualize real-time market data, system audits, and broker metrics.

Submitted by:

Aquino, Dallas A.
Buñag, Frederick Jibril L.
Carbonell, Ethan Jed V.
Corpes, Vincent L. Jr.

BS Computer Science 3A-AI

Instructor: Mr. Cyreneo S. Dofitas

Course: CCS231 - Parallel and Distributed Computing

Date: May 3, 2026

Contents

1 Introduction

1.1 System Overview

The Publish-Subscribe (Pub/Sub) messaging pattern is a fundamental architecture in distributed computing. This project implements a centralized Pub/Sub middleware system capable of handling concurrent network connections, routing messages based on hierarchical topics, and ensuring data durability. The system is composed of three primary logical tiers:

1. **External Publishers:** Independent processes that simulate and broadcast real-time data streams (e.g., cryptocurrency prices, blue-chip stocks, and system crash events).
2. **Central Broker:** The core multi-threaded server responsible for managing TCP client connections, matching published topics to subscriber patterns, and orchestrating durable queue storage.
3. **Decoupled Dashboard Backend:** A set of independent subscriber clients that ingest routed data from the broker and bridge it over WebSockets to a web-based UI for real-time visualization.

1.2 Design Goals & Principles

To fulfill the core requirements of parallel and distributed computing, the system was built upon several foundational design principles:

- **Strict Decoupling:** Publishers and subscribers are completely unaware of each other. The broker acts as an impartial router, enhancing system scalability.
- **Topic-Based Filtering:** Instead of computationally expensive content-based filtering, the broker routes messages via fast string matching using multi-level wildcards (e.g., `MARKET.CRYPTO.*`).
- **At-Least-Once Delivery Guarantee:** To handle network volatility, all messages are immediately persisted to a database before network transmission. If a subscriber disconnects, messages remain safely queued and are aggressively replayed upon reconnection.

2 High-Level Architecture

2.1 System Architecture Diagram

The architecture spans multiple network layers and processes, flowing from data generation down to web visualization. The broker operates as the central hub, utilizing an internal Thread Pool to handle concurrent socket operations seamlessly.

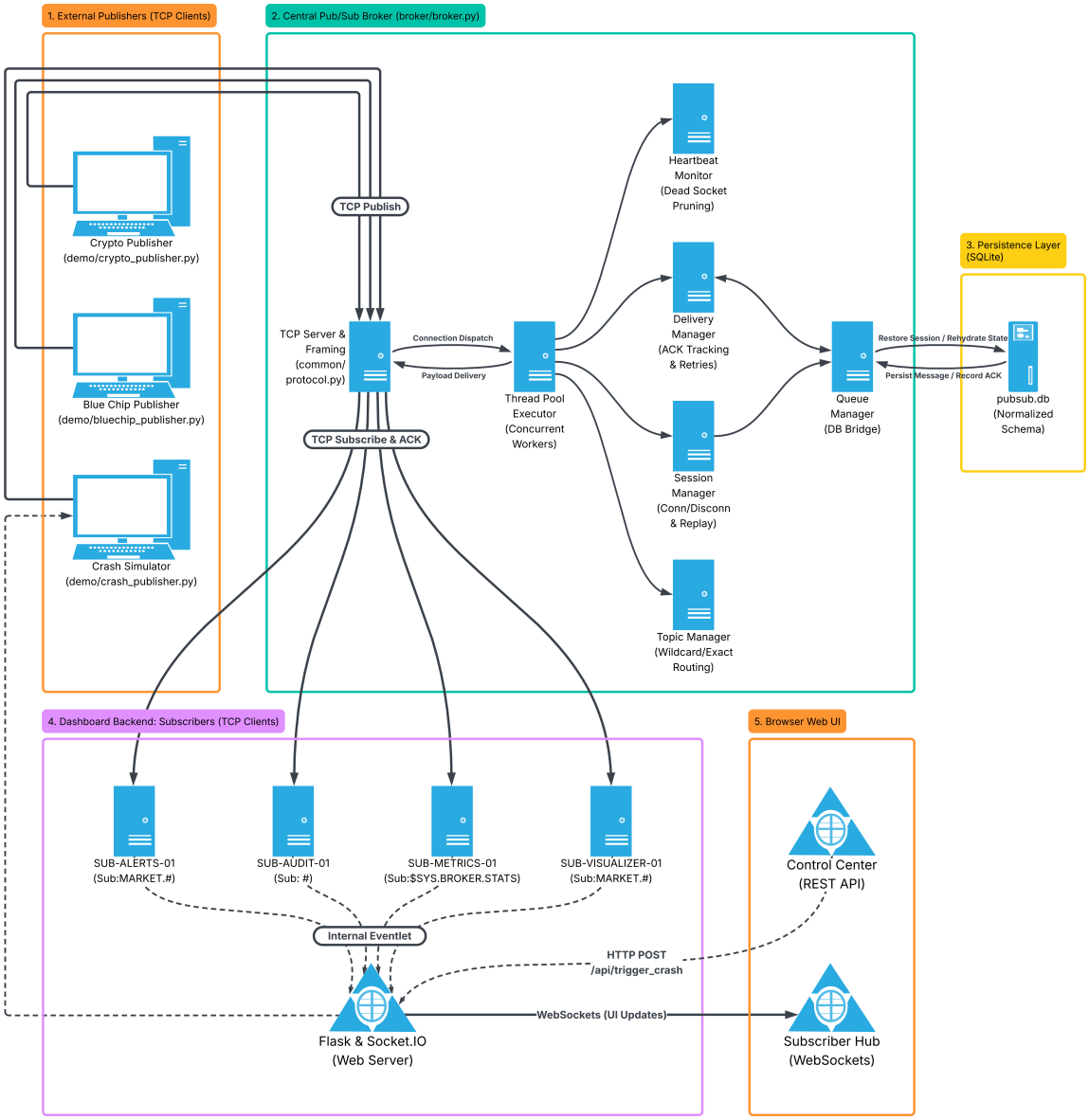

Figure 1: End-to-End System Architecture. Illustrates the flow of data from external publishers, through the multi-threaded broker components and SQLite persistence layer, into the decoupled dashboard subscribers.

As shown in Figure 1, the system avoids bottlenecks by delegating complex tasks to specialized managers within the broker. For instance, the `SessionManager` handles state resumptions, while the `DeliveryManager` interacts heavily with the `QueueManager` for disk I/O operations.

2.2 Inter-Process Communication (IPC)

In distributed systems, the underlying protocol dictating how nodes communicate is critical. This middleware utilizes raw TCP sockets for low-latency Inter-Process Communication.

However, because TCP is a continuous stream of bytes rather than discrete packets, large JSON payloads run the risk of fragmentation (where a payload arrives in multiple chunks). To mitigate this, a custom **Length-Prefix Framing Protocol** was implemented:

```
# 1. Serialize to JSON and encode to bytes
json_data = json.dumps(message.to_dict())
payload_bytes = json_data.encode('utf-8')

# 2. Pack the length of the payload into a 4-byte big-endian integer ('!I')
header = struct.pack('!I', len(payload_bytes))

# 3. Send exactly the 4-byte header followed by the payload over the socket
sock.sendall(header + payload_bytes)
```

Listing 1: Custom TCP Framing Implementation (`common/protocol.py`)

Every outgoing payload is prefixed with a 4-byte binary integer declaring its exact size. Upon receiving data, the TCP socket reader first safely reads the 4-byte header, determines the incoming payload size, and loops until the exact amount of bytes is pulled from the network buffer. This ensures 100% safe deserialization of JSON objects regardless of network latency or stream fragmentation.

2.3 Routing Mechanism: Topic-Based vs. Content-Based Filtering

A critical architectural decision in Publish-Subscribe systems is how the broker routes messages to subscribers.

- **Content-Based Filtering:** In this model, subscriptions are defined by queries over the message attributes (e.g., `price > 200 AND volume > 1000`). While highly flexible for the subscriber, it requires the broker to deserialize, parse, and evaluate the JSON payload of every incoming message against complex rulesets. This introduces immense CPU overhead and severely limits system throughput.
- **Topic-Based Filtering (Chosen Approach):** Subscriptions are based on logical channels or hierarchical strings (e.g., MARKET . CRYPTO . BTC). The broker routes messages using fast, memory-efficient string matching and wildcard resolution (* and #). By adopting Topic-Based filtering, our broker acts purely as a fast router. It remains completely agnostic to the actual payload contents, ensuring maximum scalability and minimal processing latency per message.

3 Core System Components

3.1 The Central Broker & Thread Pool

The core of the middleware is the central broker (`broker.py`). Because distributed systems must handle multiple clients simultaneously without blocking, the broker implements a concurrent thread-per-client model utilizing Python's `ThreadPoolExecutor`. To prevent memory exhaustion under high load, the thread pool is capped at a maximum number of concurrent workers.

The broker delegates specific responsibilities to independent manager classes to maintain a clean separation of concerns:

- **`TopicManager`:** Maintains in-memory sets mapping clients to their subscribed topics. It performs real-time string evaluation to support hierarchical wildcard matching (e.g., resolving `MARKET.CRYPTO.*` to `MARKET.CRYPTO.BTC`).
- **`SessionManager`:** Tracks active sockets. When a known client reconnects, it immediately triggers the `QueueManager` to replay any missed messages.
- **`HeartbeatMonitor`:** A background daemon that tracks periodic ping messages from clients. If a client misses multiple consecutive heartbeats, the monitor aggressively closes the stale TCP socket to free up thread resources.

Concurrency Control and Thread Safety (OS Support)

Because the Thread Pool allows multiple client connections to be handled concurrently, shared data structures within the broker are highly susceptible to race conditions. To enforce thread safety, the system leverages OS-level synchronization primitives. For example, the `TopicManager` utilizes Re-entrant Locks (`threading.RLock()`) when modifying subscription dictionaries. Similarly, the `HeartbeatMonitor` and `SessionManager` use standard Mutexes (`threading.Lock()`) to safely iterate over the pool of active client sockets without encountering dictionary-changed-during-iteration exceptions.

3.2 External Publishers & Control Center

Publishers are standalone Python processes that generate data and push it to the broker. The system features two continuous simulators: the **Crypto Publisher** (simulating highly volatile digital assets) and the **Blue Chip Publisher** (simulating stable market equities with mean reversion).

Additionally, the system provides a web-based **Control Center** that leverages Remote Invocation principles via a REST API. When an administrator triggers a crash event, the Flask backend receives the HTTP remote invocation and utilizes OS-level process management (`subprocess.Popen`) to dynamically spawn an independent `crash_publisher.py` background process. This script connects via TCP, fires the massive negative price spike, and immediately disconnects, thereby testing the system's edge-triggered alert logic without blocking the dashboard's main event loop.

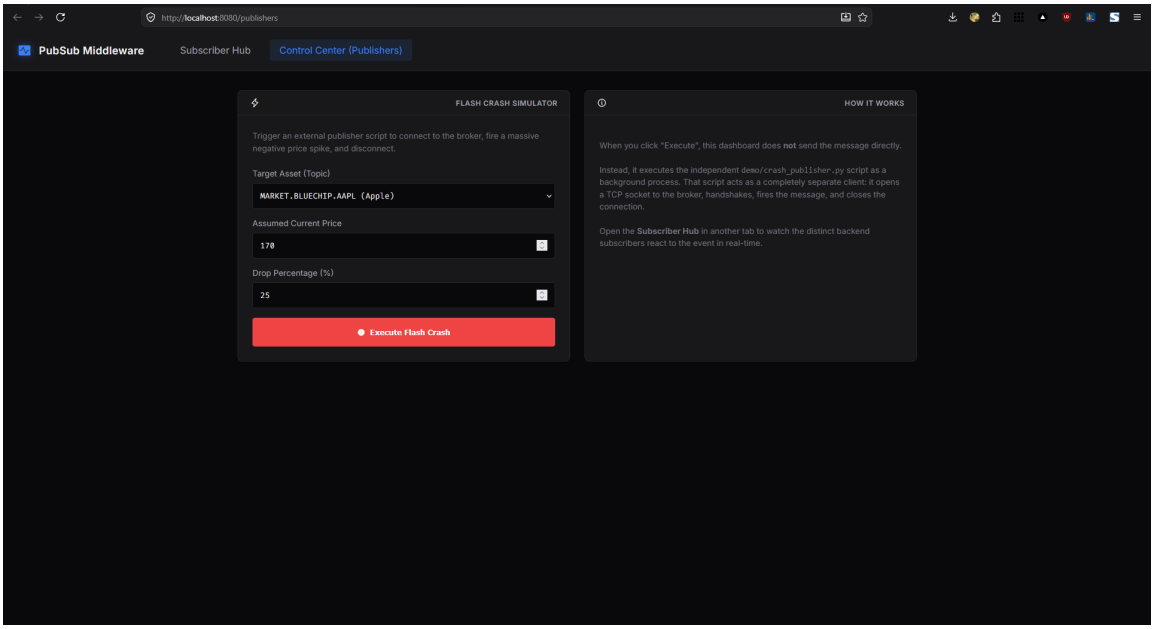

Figure 2: Publisher Control Center. This interface utilizes a REST API to dynamically spawn an independent `crash_publisher.py` process, simulating a sudden market drop to test the system's edge-triggered alert logic.

In Figure 2, when a crash is executed, the dashboard does not publish the message directly. Instead, it spawns a completely separate publisher client that connects via TCP, handshakes with the broker, fires the massive negative price spike, and immediately disconnects.

3.3 Dashboard Backend (Subscribers)

To visualize the data flowing through the broker, the web dashboard server (`app.py`) internally spawns four distinct TCP subscriber clients. These backend subscribers operate independently on background threads (using `eventlet`) and forward received messages to the browser via asynchronous WebSockets.

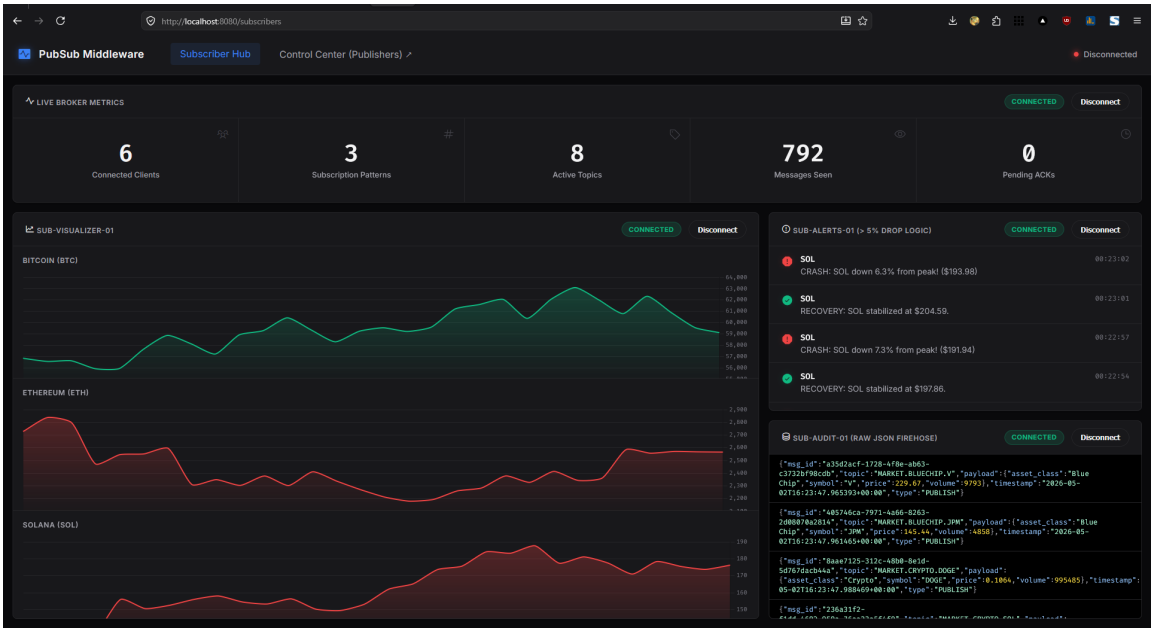

Figure 3: Subscriber Hub Web Interface. Displays real-time data bridged from the four independent backend TCP subscribers, including live charts, anomaly alerts, broker metrics, and a raw JSON firehose.

The four dedicated subscribers mapped to the UI in Figure 3 include:

1. **`SUB-VISUALIZER-01`:** Subscribes to `MARKET.#` and maps streaming price data to dynamic line charts.
2. **`SUB-ALERTS-01`:** Subscribes to `MARKET.#` and maintains high-water marks for all assets. It utilizes *edge-triggered logic* to emit critical alerts only when an asset drops more than 5% from its peak, preventing console spam.
3. **`SUB-AUDIT-01`:** Subscribes to the global wildcard `#` to capture and display the unadulterated JSON firehose for system auditing.
4. **`SUB-METRICS-01`:** Subscribes to the internal `$SYS.BROKER.STATS` topic to display live metrics such as pending ACKs and active topic counts.

4 Persistence and Data Modeling

4.1 Normalized Database Schema

To fulfill the At-Least-Once delivery guarantee, the system must persist messages to disk before attempting network transmission. The persistence layer utilizes a local SQLite database configured with Write-Ahead Logging (WAL) for high-concurrency access.

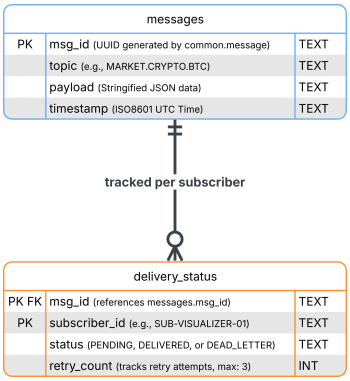

Figure 4: Entity-Relationship (ER) Diagram of the Persistence Layer. Demonstrates the normalized schema bridging the heavy message payloads with lightweight delivery status trackers.

4.2 Space Optimization Strategy

In a Pub/Sub system, a single message might need to be routed to hundreds of subscribers. Storing the entire message (which may contain a large JSON payload) for every individual subscriber would result in massive disk I/O bottlenecks and storage waste.

To solve this, the architecture employs a strictly normalized relational design (as illustrated in Figure 4):

- **`messages` Table:** Acts as the central vault. It stores the heavy JSON string (`payload`) exactly once, indexed by a universally unique identifier (`msg_id`).
- **`delivery_status` Table:** A lightweight junction table that tracks the delivery lifecycle. For every subscriber that needs the message, a tiny row is inserted here containing the state (PENDING, DELIVERED, or DEAD_LETTER) and a `retry_count`.

In this design, `msg_id` operates as a Foreign Key (ensuring referential integrity so no status exists without a valid message) and simultaneously pairs with `subscriber_id` to form a Composite Primary Key. This composite key mathematically guarantees that a specific subscriber can only have one queued state for a specific message, preventing duplicate deliveries. Once all subscribers successfully ACK the message, the `QueueManager` safely deletes the orphaned payload from the `messages` table to prevent infinite database growth.

5 Workflows and Fault Tolerance

5.1 Message Lifecycle & Session Replay

To understand how the middleware achieves resilience against network instability, it is necessary to examine the chronological execution of a message's lifecycle. The sequence diagram in Figure 5 maps the exact flow of data across the network boundary, through the broker's internal managers, into the database, and finally to the subscribers.

To provide a comprehensive view, the diagram is presented on the following page in a landscape orientation. It details three distinct operational phases: initial publication, perfect network conditions (Happy Path), and recovery from network failure (Session Replay).

5.2 At-Least-Once Delivery Mechanics

As illustrated in the sequence diagram (Figure 5), the broker relies heavily on persistent state to guarantee that no messages are lost in transit. The workflow is divided into three critical phases:

- **Phase 1: Publishing and Persistence.** When a publisher sends a payload, the broker's `TopicManager` identifies all matching subscribers. Before any attempt is made to transmit the message over the network, the `DeliveryManager` intercepts the process. It instructs the `QueueManager` to execute a database `INSERT`, saving the message with a `PENDING` status. This is the cornerstone of the At-Least-Once guarantee; if the broker loses power immediately after this step, the message remains safely stored on disk.
- **Phase 2: Happy Path Delivery (Active Client).** If a subscriber is currently connected (e.g., `SubA`), the broker pushes the message over the TCP socket. The subscriber's client application processes the data (such as updating a UI chart) and immediately replies with an ACK (Acknowledgment) packet. Upon receiving this ACK, the broker executes a DELETE query, removing the `PENDING` status from the database to conserve storage.
- **Phase 3: Fault Tolerance & Session Replay.** The true strength of the architecture is demonstrated when a subscriber drops offline (e.g., `SubB`). While disconnected, the broker continues to queue `SubB`'s messages as `PENDING` in the database. When `SubB` eventually re-establishes its TCP connection, the broker's `SessionManager` intercepts the handshake. It queries the `QueueManager` for all missed messages and instantly replays them over the new socket. Once `SubB` acknowledges them, the database is cleared.
- **Phase 4: Retry Limits & Dead Letter Queues (DLQ).** If a subscriber remains offline or continuously fails to ACK a message, the `DeliveryManager`'s background thread will periodically re-attempt delivery (up to `MAX_RETRIES`). If a message exhausts its retry limit without an acknowledgment, the database status is updated to `DEAD_LETTER`. This prevents the broker from infinitely retrying doomed messages, saving CPU cycles and network bandwidth. An administrative script (`inspect_dead_letters.py`) is provided to audit these unroutable payloads.

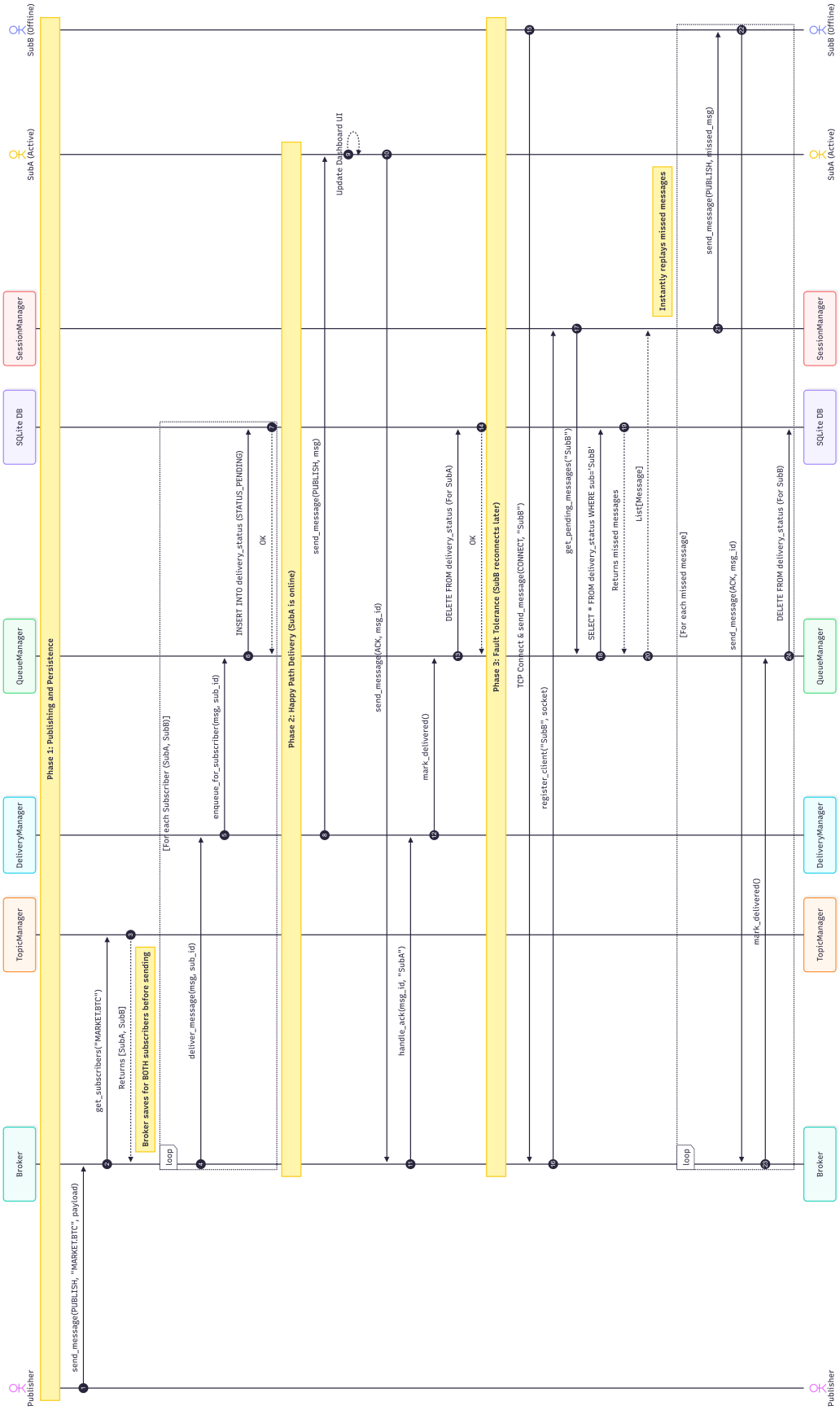

Figure 5: Message Lifecycle and Fault Tolerance Sequence Diagram. Reads top-to-bottom, illustrating the chronological execution of message publication, database persistence, happy-path delivery, and offline session replay.

6 Conclusion

This system successfully implements a robust, distributed Publish-Subscribe messaging middleware capable of handling asynchronous data streams across multiple network nodes. By decoupling data producers from consumers, the architecture ensures high scalability. Furthermore, the integration of a multi-threaded `ThreadPoolExecutor` combined with a normalized Write-Ahead Logging (WAL) SQLite persistence layer guarantees strict fault tolerance and At-Least-Once delivery without sacrificing performance.

The addition of a decoupled web dashboard effectively demonstrates how synchronous, low-level TCP communications can be seamlessly bridged into modern asynchronous WebSockets, providing real-time observability, anomaly alerting, and system auditing in a distributed environment.

Repository

The full project source code is available at:
https://github.com/Ethanolbicarbonate/Publish-Subscribe-Messaging-System

Aquino, Dallas A.
Buñag, Frederick Jibril L.
Carbonell, Ethan Jed V.
Corpes, Vincent L. Jr.